

AI ASSISTED CODING

NAME: A.charani

BATCH: 40

ROLL NO : 2303A52108

Lab 13: Code Refactoring – Improving Legacy Code with AI

Suggestions

Task Description -1 (Refactoring – Eliminating Repetitive Logic)

- Task: Use AI-assisted coding techniques to restructure a Python program that performs repeated calculations, transforming it into a reusable and maintainable design.

Instructions:

- Ask AI to locate repeated computations within the program.
- Encapsulate repeated logic into reusable functions.
- Verify that program output remains unchanged after refactoring.
- Add proper docstrings to all defined functions.

Sample Code:

```
length = 8
width = 4
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))

length = 12
width = 6
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))
```

Expected Output:

A refactored Python script that uses a single reusable function to compute area and perimeter without repeating code.

```
Lab_13.2.py > ...
1  #TASK - 1
2  #restructure a Python program that performs repeated calculations, transforming it into a reusable and maintainable design.
3  length = 8
4  width = 4
5  print("Area:", length * width)
6  print("Perimeter:", 2 * (length + width))
7  length = 12
8  width = 6
9  print("Area:", length * width)
10 print("Perimeter:", 2 * (length + width))
11 # Refactored Code:
12 def calculate_area(length, width):
13     return length * width
14 def calculate_perimeter(length, width):
15     return 2 * (length + width)
16 length = 8
17 width = 4
18 print("Area:", calculate_area(length, width))
19 print("Perimeter:", calculate_perimeter(length, width))
20 length = 12
21 width = 6
22 print("Area:", calculate_area(length, width))
23 print("Perimeter:", calculate_perimeter(length, width))
24
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS GITLENS Python De

PS C:\Users\chand\OneDrive\Documents\Github\AIAC> & 'c:\Users\chand\AppData\Local\Python\pythoncore-3.14-64\python.exe' 'c:\Users\chand\...v

○ Perimeter: 24
Area: 72
Perimeter: 36
Area: 32
Perimeter: 24
Area: 72
Perimeter: 36

ANALYSIS

- The program calculates the area and perimeter of a rectangle.
- Initially, the same calculations are repeated in the code.
- Functions `calculate_area()` and `calculate_perimeter()` are created to avoid repetition.
- This makes the code reusable, cleaner, and easier to maintain.

Task Description -2 (Refactoring – Modularizing Inline

Calculations)

- Task: Use AI to refactor a Python script by extracting repeated inline calculations into reusable and well-documented functions.

Instructions:

- Identify repeated calculation patterns in the code.
- Convert them into reusable functions.
- Ensure readability and proper documentation.

Sample Code:

```
amount = 1000
```

```
discount = amount * 0.10
```

```
final_amount = amount - discount
```

```
print("Final Amount:", final_amount)
```

```
amount = 2000
```

```
discount = amount * 0.10
```

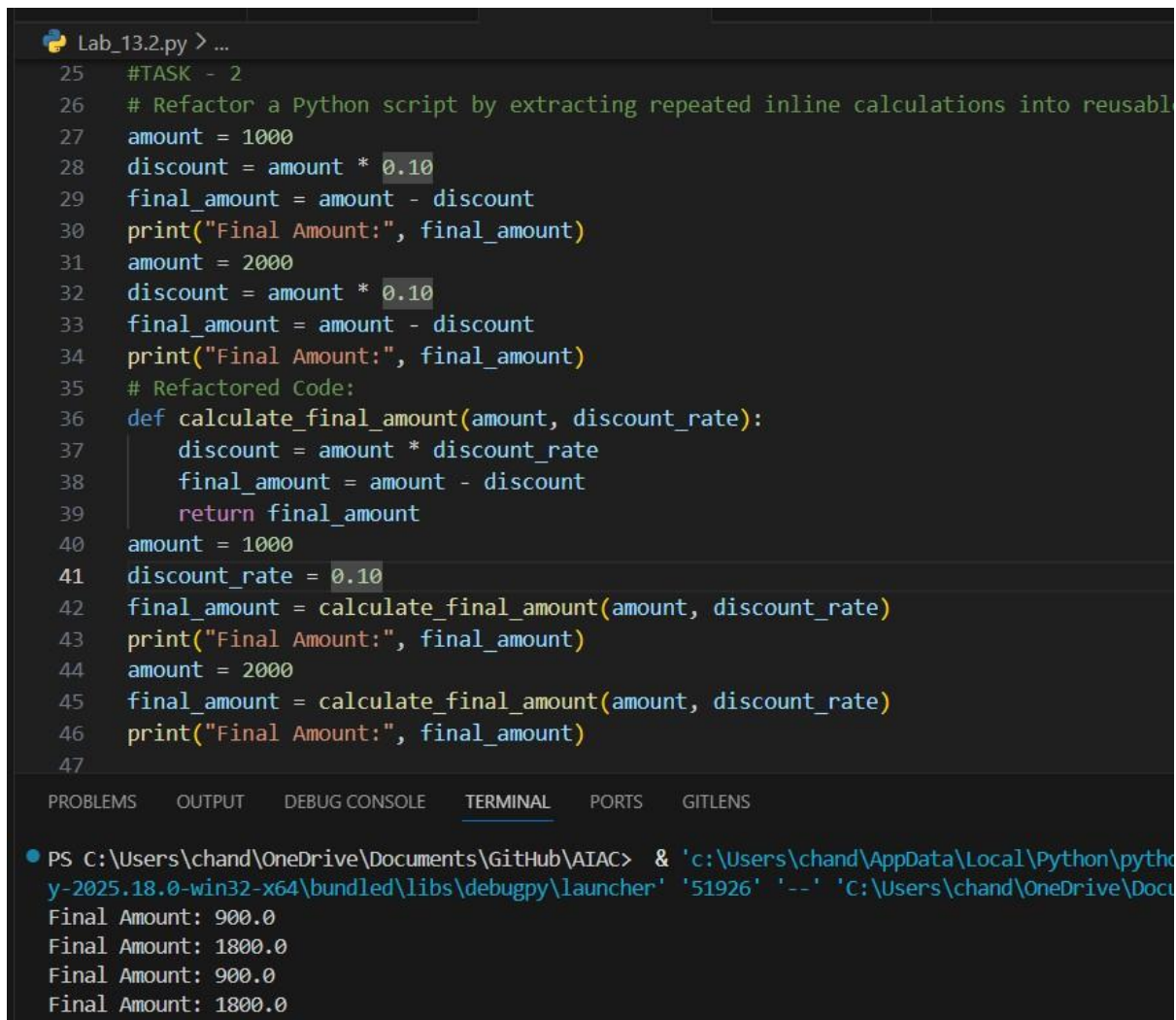
```
final_amount = amount - discount
```

```
print("Final Amount:", final_amount)
```

Expected Output:

A modular Python program that uses a reusable function to compute

final amounts for different inputs.



The screenshot shows a Python IDE with a file named 'Lab_13.2.py'. The code is divided into two sections. The first section (lines 25-34) shows the original code with repeated calculations for two different amounts (1000 and 2000). The second section (lines 35-46) shows the refactored code, which introduces a function 'calculate_final_amount' that takes 'amount' and 'discount_rate' as arguments. The function calculates the discount and the final amount, then returns the final amount. The main code then calls this function for both amounts. The terminal output at the bottom shows the execution results: 'Final Amount: 900.0' for the first case and 'Final Amount: 1800.0' for the second case.

```
25 #TASK - 2
26 # Refactor a Python script by extracting repeated inline calculations into reusable
27 amount = 1000
28 discount = amount * 0.10
29 final_amount = amount - discount
30 print("Final Amount:", final_amount)
31 amount = 2000
32 discount = amount * 0.10
33 final_amount = amount - discount
34 print("Final Amount:", final_amount)
35 # Refactored Code:
36 def calculate_final_amount(amount, discount_rate):
37     discount = amount * discount_rate
38     final_amount = amount - discount
39     return final_amount
40 amount = 1000
41 discount_rate = 0.10
42 final_amount = calculate_final_amount(amount, discount_rate)
43 print("Final Amount:", final_amount)
44 amount = 2000
45 final_amount = calculate_final_amount(amount, discount_rate)
46 print("Final Amount:", final_amount)
47
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

```
PS C:\Users\chand\OneDrive\Documents\GitHub\AIAC> & 'c:\Users\chand\AppData\Local\Python\python\python.exe' -2025.18.0-win32-x64\bundled\libs\debugpy\launcher '51926' '--' 'c:\Users\chand\OneDrive\Documents\GitHub\AIAC\Lab_13.2.py'
Final Amount: 900.0
Final Amount: 1800.0
Final Amount: 900.0
Final Amount: 1800.0
```

ANALYSIS

-
- The program calculates the final amount after a discount.
 - Initially, the same discount calculation is repeated in the code.
 - A function `calculate_final_amount()` is created to avoid repetition.
 - This makes the code cleaner, reusable, and easier to maintain.
-

Task Description -3 (Refactoring – Improving Loop and

Conditional Performance)

- Task: Apply AI-based suggestions to enhance the performance of Python code that relies on nested loops and repeated conditional checks.

Instructions:

- Use AI to recommend more efficient data structures or logic simplifications.
- Replace inefficient constructs while preserving program behavior.
- Compare execution efficiency before and after refactoring.

Sample Code:

```
students = ["Anil", "Bhavya", "Charan", "Divya"]
```

```
check_list = ["Charan", "Esha", "Bhavya"]
```

```
for name in check_list:
```

```
    exists = False
```

```
    for student in students:
```

```
        if name == student:
```

```
            exists = True
```

```
    if exists:
```

```
        print(name, "found")
```

```
    else:
```

```
        print(name, "not found")
```

Expected Output:

An optimized Python program that performs faster lookups using

efficient logic, along with execution performance comparison.

```
Lab_13.2.py > ...
47
48 #TASK - 3
49 #Apply AI-based suggestions to enhance the performance of Python code that relies on nested loops
50 '''students = ["Anil", "Bhavya", "Charan", "Divya"]
51 check_list = ["Charan", "Esha", "Bhavya"]
52 for name in check_list:
53     exists = False
54     for student in students:
55         if name == student:
56             exists = True
57         if exists:
58             print(name, "found")
59         else:
60             print(name, "not found")'''
61 # Refactored Code:
62 students = ["Anil", "Bhavya", "Charan", "Divya"]
63 check_list = ["Charan", "Esha", "Bhavya"]
64 for name in check_list:
65     if name in students:
66         print(name, "found")
67     else:
68         print(name, "not found")
69
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS GITLENS

```
PS C:\Users\chand\OneDrive\Documents\GitHub\AIAC> & 'c:\Users\chand\AppData\Local\Python\pythoncore-3.14-
y-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '58672' '--' 'C:\Users\chand\OneDrive\Documents\GitH
Charan found
Esha not found
Bhavya found
PS C:\Users\chand\OneDrive\Documents\GitHub\AIAC>
```

ANALYSIS

- The program checks whether names in `check_list` exist in the students list.
- Initially, it uses nested loops, which is slower.
- The refactored code uses `if name in students` to check directly.
- This improves performance and makes the code simpler.

Task Description -4 (Refactoring – Replacing Hardcoded Values with Constants)

- Task: Utilize AI assistance to improve maintainability by replacing hardcoded numerical values with named constants.

Instructions:

- Detect hardcoded numeric values used directly in expressions.

- Define descriptive constants at the beginning of the program.
- Update calculations to use these constants without changing results.

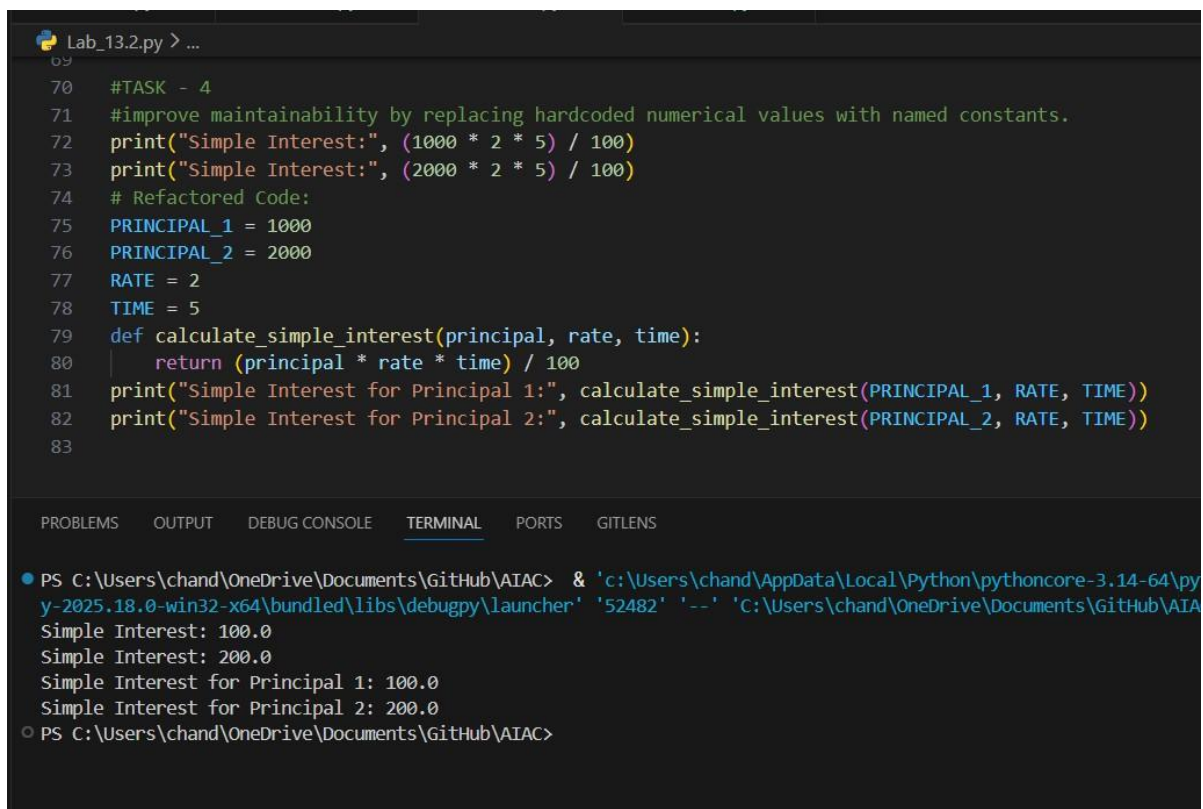
Sample Code:

```
print("Simple Interest:", (1000 * 2 * 5) / 100)
```

```
print("Simple Interest:", (2000 * 2 * 5) / 100)
```

Expected Output:

A revised Python program using constants such as RATE and TIME for clearer and maintainable calculations..



The screenshot shows a Python IDE with a file named 'Lab_13.2.py'. The code is as follows:

```
70 #TASK - 4
71 #improve maintainability by replacing hardcoded numerical values with named constants.
72 print("Simple Interest:", (1000 * 2 * 5) / 100)
73 print("Simple Interest:", (2000 * 2 * 5) / 100)
74 # Refactored Code:
75 PRINCIPAL_1 = 1000
76 PRINCIPAL_2 = 2000
77 RATE = 2
78 TIME = 5
79 def calculate_simple_interest(principal, rate, time):
80     return (principal * rate * time) / 100
81 print("Simple Interest for Principal 1:", calculate_simple_interest(PRINCIPAL_1, RATE, TIME))
82 print("Simple Interest for Principal 2:", calculate_simple_interest(PRINCIPAL_2, RATE, TIME))
83
```

The terminal output shows the execution of the program:

```
PS C:\Users\chand\OneDrive\Documents\GitHub\AIAC> & 'c:\Users\chand\AppData\Local\Python\pythoncore-3.14-64\python-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '52482' '--' 'C:\Users\chand\OneDrive\Documents\GitHub\AIAC\Lab_13.2.py'
Simple Interest: 100.0
Simple Interest: 200.0
Simple Interest for Principal 1: 100.0
Simple Interest for Principal 2: 200.0
PS C:\Users\chand\OneDrive\Documents\GitHub\AIAC>
```

ANALYSIS

- The program calculates **Simple Interest**.
- Initially, it uses **hardcoded values** in the formula.
- The refactored code uses **named constants and a function** `calculate_simple_interest()`.
- This makes the code **clearer, reusable, and easier to maintain**.

Task Description -5 (Re factoring – Enhancing Variable Naming and Code Clarity)

- Task: Apply AI tools to improve code readability by renaming unclear variables and adding meaningful comments..

Instructions:

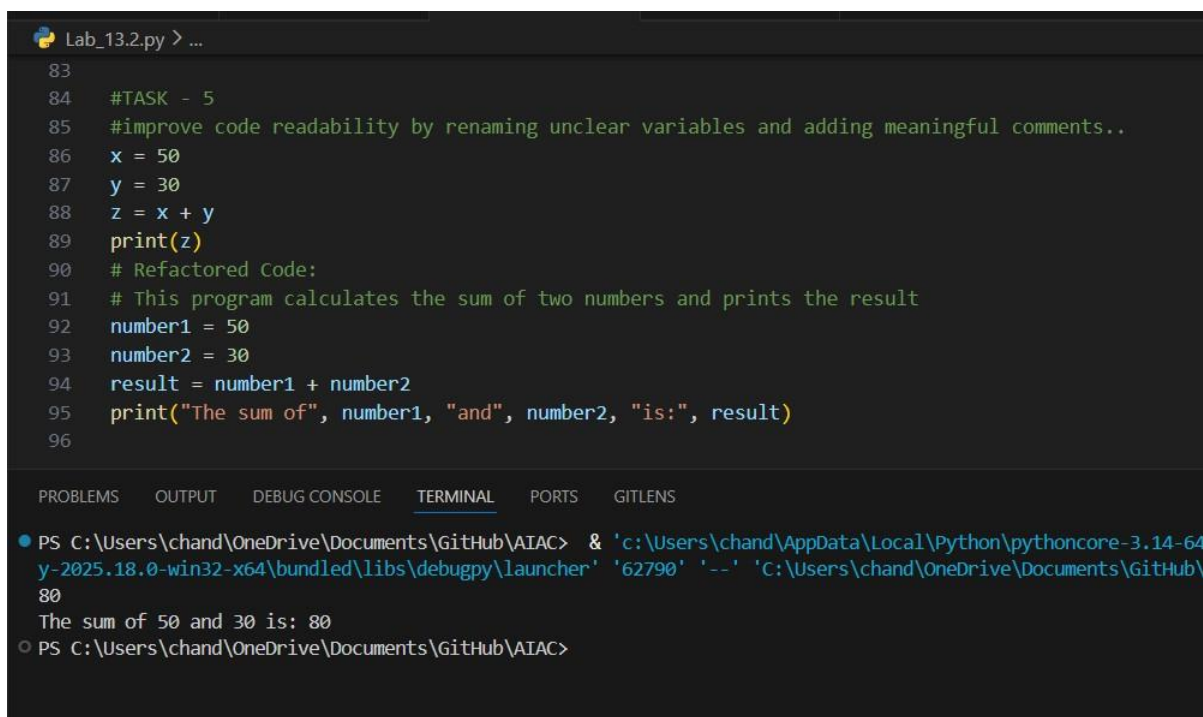
- Replace non-descriptive variable names with meaningful identifiers.
- Add inline comments where the logic may be unclear.
- Ensure program output remains the same.

Sample Code:

```
x = 50  
y = 30  
z = x + y  
print(z)
```

Expected Output:

A Python program with clear variable names and explanatory comments that improve readability without altering functionality.



The screenshot shows a code editor with a file named 'Lab_13.2.py'. The code is as follows:

```
83  
84 #TASK - 5  
85 #improve code readability by renaming unclear variables and adding meaningful comments..  
86 x = 50  
87 y = 30  
88 z = x + y  
89 print(z)  
90 # Refactored Code:  
91 # This program calculates the sum of two numbers and prints the result  
92 number1 = 50  
93 number2 = 30  
94 result = number1 + number2  
95 print("The sum of", number1, "and", number2, "is:", result)  
96
```

Below the code editor, the terminal output is shown:

```
PS C:\Users\chand\OneDrive\Documents\GitHub\AIAC> & 'c:\Users\chand\AppData\Local\Python\pythoncore-3.14-64  
y-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '62790' '--' 'C:\Users\chand\OneDrive\Documents\GitHub\  
80  
The sum of 50 and 30 is: 80  
PS C:\Users\chand\OneDrive\Documents\GitHub\AIAC>
```

ANALYSIS

- The program **adds two numbers and prints the result**.
 - Initially, **unclear variable names** (x, y, z) are used.
 - The refactored code uses **meaningful variable names** and comments.
 - This **improves readability and understanding of the code**.
-